Difference between cudaMallocManaged and cudaMallocHost

> cuda

user20205 Mar 15 '22

Hey,
I'm pretty confused about the difference between allocating memory with cudaMallocHost and with cudaMallocManaged.
I'm using NVIDIA GeForce RTX 2080 SUPER GPU.
As I read in this **article** , cudaMallocManaged, first allocate the size of bytes in the device memory.
So, according to this article, when I'm trying to access this memory from the CPU, a page fault occurs, and the GPU driver migrates the page from device memory to CPU memory.
So, basically, when I'm trying to access this memory, it copies it to the host memory, and of course, also the opposite happens.
cudaMallocHost, according to Cuda runtime API documentation, allocates host memory that is page-locked and accessible to the device.
"The driver tracks the virtual memory ranges allocated with this function and automatically accelerates calls to functions such as cudaMemcpy."
So, basically, when I'm trying to access memory allocated by cudaHostMalloc in device code, it copies it to the device memory.
If all the information I mentioned above is true, it seems like cudaMallocHost and cudaMallocManaged are doing the same. Am I wrong?

> ☑ Solved by **Robert_Crovella** in **post #2**
>
> No, it doesn't. The data will be copied to the device, but not to device memory. They are not doing the same thing in the general/typical case. Managed memory (cudaMallocManaged) moves the resident location of an allocation to the processor that needs it. Pinned memory does not. Let's take an ...

Robert_Crovella Moderator Mar 15 '22

> user20205:
>
> So, basically, when I'm trying to access memory allocated by cudaHostMalloc in device code, it copies it to the device memory.

No, it doesn't. The data will be copied to the device, but not to device memory.

They are not doing the same thing in the general/typical case.

Managed memory (cudaMallocManaged) *moves* the resident location of an allocation to the processor that needs it. Pinned memory does not. Let's take an example. You have an allocation of 4k bytes using `cudaMallocManaged`. You fill that allocation in host code. While your host code is filling it, the allocation is "resident" in host/CPU memory, similar to an ordinary allocation made with `new` or `malloc`.

Later, you launch a kernel and pass the pointer to that allocation to your kernel code. Your kernel code begins to access data within that 4k page. At the first access (or at kernel-launch in the pre-pascal regime) this 4k page will be "migrated" from host to device. Your code may have only "touched" one byte in that page (so far) but at first "touch", the entire page is copied from host to device, and becomes "resident" in device memory. Subsequent accesses to any data in that page from kernel code will be fulfilled from device memory, at device memory speeds (typically several hundred GB/s access bandwidth).

Now lets consider the pinned memory case (cudaHostAlloc/cudaMallocHost). You have an allocation of 4k bytes using `cudaMallocHost`. You fill that allocation in host code. While your host code is filling it, the allocation is "resident" in host/CPU memory, similar to an ordinary allocation made with `new` or `malloc`. So far, no difference.

Later, you launch a kernel and pass the pointer to that allocation to your kernel code. Your kernel code begins to access data within that 4k page. In order to support this access, a "mapping" is made so that you can access data that is still resident in host/CPU memory, in device code. There is no en-masse movement of the data from host to device, and the allocation never becomes "resident" in device memory. Very specifically, let's say your kernel code requests the first element (or line) in the allocation. At that moment, what will happen is the first element (or line) will be transferred from host memory and delivered to the thread/warp that is requesting that data. Now let's suppose later that another thread/warp, in another SM, requests the same data. The data will be transferred **again** from host to device, to satisfy the needs of the device code. Each of these transfers will happen at PCIE speeds (e.g. ~10-20GB/s) not at device memory speeds (~hundreds of GB/s). Now let's suppose another warp needs the data. Its not resident in device memory, so it will **again** be transferred, from host to device, over the PCIE bus.

For large-scale activity, especially in the case of repeated access, the managed memory case gives the possibility to kernel code to access the data at device memory speeds (ignoring, or after the initial bulk transfer of the data). If we ignore L1 caching effects, the pinned memory case will always be accessed at PCIE bus speeds (which are generally much slower), from device code.

You can get an orderly introduction to CUDA **here** . A deeper treatment of pinned memory is available in section 7 there, and a deeper treatment of managed memory is available in section 6 there.

user20205 Mar 16 '22

Thank you for the informative answer!

Closed on Mar 30, 2022

This topic was automatically closed 14 days after the last reply. New replies are no longer allowed.

New & Unread Topics

Topic	Replies	Views	Activity
Async thrust operation launches appear serially processed in nsight systems cuda performance parallel-computing	4	8	2h
☑ **[q] mask in __shfl_sync()** cuda kernel	3	9	3h
Potential memory leak - compute-sanitizer shows nothing camera cuda jetson	6	36	3h
Is there anyway to query the number of remaining operations in a stream?	2	5	4h
How to seperate declare header and implement files in cuda programming? cuda	6	16	4h
What is a transaction from HBM to L2?	2	7	5h
How to use nvrtc && nvjit? cuda	1	7	6h
What's the difference between CUDA stack and local memory?	1	8	9h
It seems that CUDA kernel are not running parallel	5	11	11h
How to use the `ex2.approx.f16x2` instruction?	2	27	Aug 29

There are 190 new topics remaining, or browse other topics in